

GPU 编程：从向量加法到分块矩阵乘

目 录

1 引言：为什么需要 GPU 编程	2
2 SIMT 架构与 CUDA 编程模型	2
2.1 SIMT 硬件：一个 GPU 里有什么	2
2.2 CUDA 编程模型：Grid、Block、Warp、Thread	3
2.3 实战：把串行 Vector Add 搬到 GPU	3
2.4 让一个 Thread 处理多个元素	4
2.5 完整的 CUDA 程序	5
2.6 CUDA Runtime API	5
2.7 Memory Coalescing：让 Warp 访问连续地址	6
2.8 Warp Divergence：分支带来的浪费	6
2.9 本节小结	7
3 Tensor Core 与分块矩阵乘	7
3.1 从 Tiled GEMM 说起	7
3.2 一个 Block 对应一个 C Tile	7
3.3 存储层次与 K 循环	7
3.4 Tensor Core 指令：MMA (Volta/Ampere)	8
3.5 WGMMMA (Hopper)	8
3.6 TCGen05 + TMEM (Blackwell)	9
4 数据搬运与排布	9
4.1 ldmatrix：SMEM → Register Fragment	9
4.2 Bank Conflict：Shared Memory 的隐藏陷阱	10
4.3 XOR Swizzle：打乱布局消除冲突	10
4.4 cp.async：绕过中间寄存器	10
4.5 Double Buffer：空间换重叠	11
4.6 TMA：一个 Thread 搬运整个 Tile	11
4.7 mbarrier 与 Warp Specialization	11
5 用 TileLang 表达分块矩阵乘	12
6 本章你将学会	13
7 要点速查	13
8 小结	14

1 引言：为什么需要 GPU 编程

我们已经熟悉 CPU 上的多线程并行，为什么还要专门学 GPU 编程？因为 CPU 的核心数量有限（几十个量级），而一颗 GPU 拥有数千个并行计算单元，天然适合“把同一个简单操作重复执行千万次”的工作负载。深度学习的矩阵乘、物理模拟的逐元素更新、图像处理的逐像素运算，都属于这类**数据并行**（Data Parallelism）任务。

但 GPU 的并行模型和 CPU 截然不同：CPU 追求单线程的复杂控制与低延迟，GPU 则用大量轻量线程掩盖访存延迟。要把算法搬到 GPU 上，我们必须理解它的硬件架构和编程模型，否则写出的代码可能比 CPU 还慢。

直觉 你可以把 CPU 想象成“几个全能博士”，每个都能独立处理复杂任务；GPU 则是“成千上万个流水线工人”，每个人都只会做一件简单的事，但胜在人多、步调一致。要让这群工人高效工作，关键是把任务切得足够细，并且让他们齐步走。



Figure 1: 大规模并行的视觉隐喻（AI 生成装饰图，不表达具体硬件结构）

本章从最简单的 **Vector Add**（向量加法）出发，逐步走到现代 GPU 上的 **Tiled GEMM**（分块矩阵乘），覆盖四条主线：SIMT 硬件架构与 CUDA 编程模型、Tensor Core 的演进、数据搬运与布局、以及用 TileLang 表达分块计算。

2 SIMT 架构与 CUDA 编程模型

2.1 SIMT 硬件：一个 GPU 里有什么

要理解怎么编程，先得看清硬件。一颗 GPU 由若干个 **SM**（Streaming Multiprocessor，流式多处理器）组成，每个 SM 内部有大量 **CUDA Core**（CUDA 核心，执行标量算术的单

元)、**寄存器** (Registers)、和一块 **Shared Memory** (共享内存, SM 私有的高速 SRAM)。所有 SM 共享一片大容量的 **Global Memory** (全局内存, 即显存 HBM)。

直觉 关键概念是 **Warp** (线程束): 32 个 Thread 组成一个 Warp, 它们共享同一条指令流。换句话说, 控制单元一次发射一条指令, 32 个 Thread 同时执行这条指令, 只是各自操作自己寄存器里的不同数据。这就是 **SIMT** (Single Instruction, Multiple Threads, 单指令多线程) 的含义: 指令相同, 数据各异。

这与 CPU 的 **SIMD** (单指令多数据) 类似但不同。SIMD 是一条指令显式操作一个向量寄存器 (比如 8 个 float); SIMT 则是每个 Thread 看起来像独立执行标量代码, 硬件在底层把 32 个标量线程捆绑成一条向量指令。对程序员来说, 写 CUDA 代码就像写普通的标量循环, 比手写 SIMD 内建函数更友好。

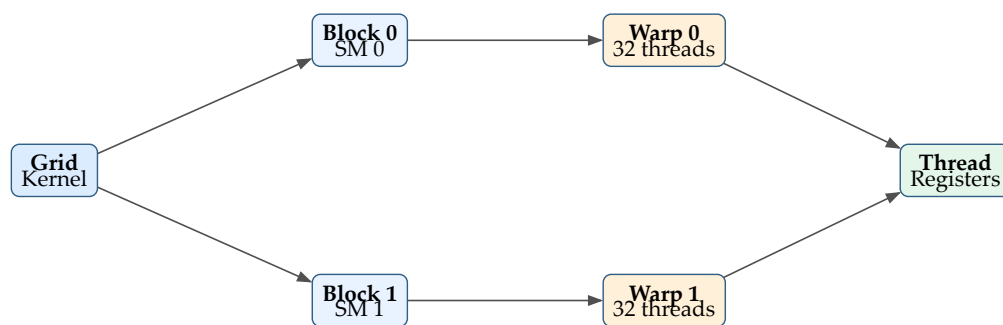
2.2 CUDA 编程模型: Grid、Block、Warp、Thread

硬件层次对应一套编程抽象: 一个 Kernel (在 GPU 上执行的函数) 启动时形成一个 **Grid** (网格), Grid 由若干 **Thread Block** (线程块, 简称 Block) 组成, 每个 Block 被调度到某个 SM 上执行, Block 内部又分成若干 Warp, 最终到 Thread。

1. GPU 承载整个 Kernel 的 Grid。
2. SM 是一个 Thread Block 的执行位置。
3. CUDA Core 执行单个 Thread 的算术指令。

层级对应关系是理解 GPU 优化的钥匙: Block 内的 Thread 可以通过 Shared Memory 协作与同步, 但不同 Block 之间默认无法直接通信。这决定了我们如何切分数据。

Figure 2 把软件抽象与调度位置串在一起。需要特别注意, 图中的箭头表示层级和调度关系, 不表示 Grid、Block、Warp 会在运行时依次创建。



依据课程原稿第 4 页重绘; Fletcher 0.5.8

Figure 2: CUDA 执行层次与硬件调度关系

2.3 实战: 把串行 Vector Add 搬到 GPU

我们先看 CPU 上的串行向量加法, 再把它改成 GPU 版本。CPU 版本一个 Thread 依次处理下标 $i = 0, 1, \dots, N - 1$:

```
for (int i = 0; i < N; ++i) {  
    C[i] = A[i] + B[i];  
}
```

GPU 版本的思路是：让成千上万个 Thread 各自认领一个下标 i ，同时算 $C[i] = A[i] + B[i]$ 。关键在于每个 Thread 如何算出自己的下标。Block 在 Grid 里有编号 `blockIdx.x`，每个 Block 内 Thread 有编号 `threadIdx.x`，每个 Block 的大小是 `blockDim.x`，于是全局下标为：

```
__global__ void vector_add(
    const float* A,
    const float* B,
    float* C,
    int N
) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        C[i] = A[i] + B[i];
    }
}
```

逐行批注：__global__ 修饰符声明这是一个可被 CPU 调用、在 GPU 上执行的 Kernel。
`blockIdx.x * blockDim.x + threadIdx.x` 把“第几个 Block”和“Block 内第几个 Thread”拼成全局下标。`if (i < N)` 是边界保护，因为启动的 Thread 总数往往不是 N 的整数倍，多出来的 Thread 会被闲置。

例 取 $N = 1024$ ，启动 4 个 Block、每个 256 个 Thread，共 $4 \times 256 = 1024$ 个 Thread，恰好一个 Thread 算一个元素。第 1 个 Block 里第 50 个 Thread 的全局下标是 $1 \times 256 + 50 = 306$ ，它计算 $C[306] = A[306] + B[306]$ 。

2.4 让一个 Thread 处理多个元素

当 N 很大时，为每个元素开一个 Thread 不现实。我们让一个 Thread 处理多个元素，用 **stride**（步长）跳跃前进：每个 Thread 依次处理 i 、 $i + \text{stride}$ 、 $i + 2 \text{ stride}$ ，直到覆盖整个数组，其中 $\text{stride} = \text{blockDim.x} \times \text{gridDim.x}$ （所有 Thread 的总数）。

```
__global__ void vector_add(
    const float* A, const float* B,
    float* C, int N
) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x * gridDim.x;
    for (; i < N; i += stride) {
        C[i] = A[i] + B[i];
    }
}
```

启动时在 Host（CPU）端用 `<<<blocks, threads>>>` 语法设定 Grid 和 Block 的大小并提交 Kernel：

```
int threads = 256;
int block_stride = threads * elem_per_thread;
int blocks = ceil_div(N, block_stride);

vector_add<<<blocks, threads>>>>(A, B, C, N);
```

```
// or use fixed grid size
vector_add<<<64, 256>>>>(A, B, C, N);
```

2.5 完整的 CUDA 程序

一个能运行的 CUDA 程序包含 Host 代码和 Device 代码两部分。Device 代码是我们上面写的 Kernel，Host 代码负责分配显存、搬运数据、启动 Kernel、等结果、回收显存：

```
#include <cuda_runtime.h>
#include <cstdio>
#include <vector>

int main() {
    const int N = 1 << 20;
    const size_t bytes = size_t(N) * sizeof(float);
    std::vector<float> h_A(N, 1.0f), h_B(N, 2.0f), h_C(N);
    float *d_A = nullptr, *d_B = nullptr, *d_C = nullptr;
    cudaMalloc(&d_A, bytes);
    cudaMalloc(&d_B, bytes);
    cudaMalloc(&d_C, bytes);
    cudaMemcpy(d_A, h_A.data(), bytes, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B.data(), bytes, cudaMemcpyHostToDevice);
    vector_add<<<64, 256>>>>(d_A, d_B, d_C, N);
    cudaDeviceSynchronize();
    cudaMemcpy(h_C.data(), d_C, bytes, cudaMemcpyDeviceToHost);
    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
    return 0;
}
```

逐行批注：*h_A、h_B* 是 Host 端的 *std::vector*，*d_* 前缀表示 Device 指针。*cudaMalloc* 在 GPU 全局内存分配空间并返回 Device 指针。*cudaMemcpy(..., cudaMemcpyHostToDevice)* 把输入数据从 CPU 搬到 GPU。*<<<64, 256>>>* 启动 64 个 Block、每个 256 个 Thread。*cudaDeviceSynchronize* 让 CPU 等待 GPU 把活干完。最后 *cudaMemcpy(..., cudaMemcpyDeviceToHost)* 把结果搬回 CPU，*cudaFree* 释放显存。

2.6 CUDA Runtime API

上面用到的一组 API 把 Host 内存、Device 内存和异步 Kernel 串成一条执行路径，值得单独记牢：

1. **cudaMalloc**：在 GPU 的 Global Memory 中分配空间，返回 Device 指针。
2. **cudaMemcpy**：显式搬运数据，HostToDevice 送入输入，DeviceToHost 取回结果。
3. **cudaDeviceSynchronize**：让 CPU 等待此前提交的 GPU 工作完成。
4. **cudaFree**：释放由 *cudaMalloc* 分配的 GPU 内存。

直觉 Kernel launch 默认是异步的：CPU 提交工作后立刻继续往下跑，不会等 GPU 算完。只有在同步或数据回拷等边界处，CPU 才会停下来等。这个设计让 CPU 和 GPU 可以重叠工作，但也意味着你必须显式同步，否则可能读到还没算完的结果。

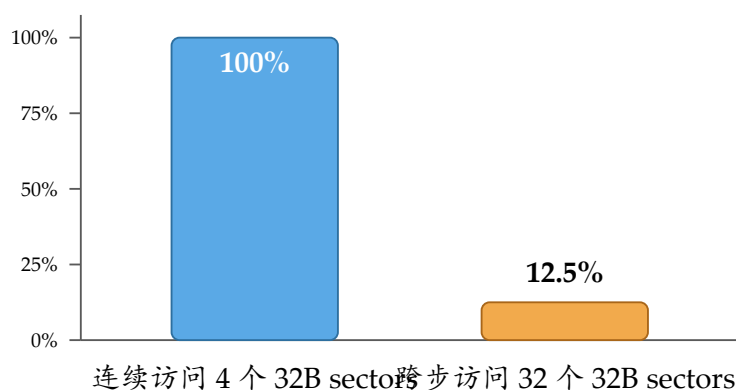
2.7 Memory Coalescing: 让 Warp 访问连续地址

GPU 访问 Global Memory 的最小单位不是单字节，而是一个连续的 32 字节 **Transaction** (事务)，称为一个 **sector**。当 Warp 内的 32 个 Thread 访问连续地址时，它们的请求可以被合并成少量事务，这叫 **Memory Coalescing** (合并访存)。

例 每个 Thread 访问一个 int (4 字节)，8 个连续 Thread 访问 $A[0]$ 到 $A[7]$ ，共 $8 \times 4 = 32$ 字节，恰好一个 32B sector，只需 1 个 transaction。这叫 **Coalesced** (合并访问)。

相反，如果 8 个 Thread 跨步访问 $A[0], A[8], A[16], A[24], \dots$ ，每个地址落在不同的 sector，就需要 8 个 transaction，搬运 256 字节却只用上 32 字节，带宽利用率仅 $\frac{1}{8}$ 。这叫 **Strided** (跨步访问)。

把例子扩展到完整 Warp 后，连续访问只需要 4 个 sector，而最坏的跨步访问需要 32 个 sector。Figure 3 展示的是由事务数量直接推导出的有效带宽比例，不是一次性能实测。



数值由课程原稿第 10 页示例推导；CeTZ 0.5.2

Figure 3: Warp 读取 32 个 int 时的有效带宽比例

实践口诀：让相邻 Thread 处理相邻任务，即 $A[\text{base} + \text{threadIdx.x}]$ ，避免按行存储的矩阵按列访问 $A[\text{base} + \text{threadIdx.x} \times \text{stride}]$ 。前者合并，后者散乱。

2.8 Warp Divergence: 分支带来的浪费

一个 Warp 共享指令流，但当不同 Thread 在 if-else 中选择不同路径时，硬件只能让所有路径依次执行。这叫 **Warp Divergence** (分支分歧)。

```
if (threadIdx.x < 16) path_A(); else path_B();
```

直觉 Warp 内前 16 个 Thread 走 path_A，后 16 个走 path_B。但 SIMT 没法同时执行两条路径，只好先让一半 Thread 执行 path_A (另一半被 mask 掉闲置)，再让另一半执行 path_B (前一半闲置)。两条路径的执行时间会相加，等于浪费了一半算力。如果整个 Warp 都走同一条路径，就没有浪费。

优化思路：尽量让同一个 Warp 内的 Thread 走相同分支。如果数据本身的分支模式与 Warp 边界对齐 (比如每 32 个元素同属一类)，就能避免分歧。

2.9 本节小结

1. **SIMT 硬件**: GPU 由多个 SM 组成, Warp 共享指令流, Thread 保留自己的寄存器。
2. **CUDA 编程模型**: Grid、Block、Warp、Thread 把问题按层级拆分。
3. **CUDA Runtime**: cudaMalloc/cudaFree、cudaMemcpy、Kernel launch、同步。
4. **优化要点**: 注意 Memory Coalescing, 避免 Warp Divergence。

下一步我们考虑更复杂的 Tiled GEMM 和 Tensor Core, 它们是现代深度学习算子的核心。

3 Tensor Core 与分块矩阵乘

3.1 从 Tiled GEMM 说起

矩阵乘法 $C = A \times B$ 是深度学习的核心计算。当矩阵很大时, 我们把它切成小块 (Tile), 一个计算单元负责一个输出 Tile, 沿着 K 维反复累加, 这叫 **Tiled GEMM** (分块矩阵乘)。它和 CPU 上的多核 Tiled GEMM 非常类似, 核心区别在于 Shared Memory 和 Tensor Core 需要特殊编程。

GPU 上的 Tiled GEMM 有清晰的层次:

1. **Block Tile**: 在 Shared Memory 中复用 A/B 子块。
2. **Warp Tile**: 把计算分配给 Warp。
3. **Thread Tile**: 调用 Tensor Core 完成小矩阵乘。
4. **Epilogue**: 合并写回 C。

3.2 一个 Block 对应一个 C Tile

关键映射关系是: 一个 **Block** 负责一个输出 **C Tile**。假设输出矩阵 C 被切成 $B_M \times B_N$ 的 Tile, 那么 Block (m, n) 负责计算 $C_{m,n}$, 它需要读取 A 的第 m 个行 Tile (大小 $B_M \times K$) 和 B 的第 n 个行 Tile (大小 $B_N \times K$), 所有 K 步的中间结果都累加到同一个 $C_{m,n}$ 上。

直觉 为什么要这样切? 因为 $C_{m,n}$ 的计算需要反复用到 A 的某一段行和 B 的某一段列。把它们一次性搬进 Shared Memory, 在片上高速 SRAM 里反复读取, 就避免了每次都去慢速显存里取。这就是“搬一次、用多次”的复用思想。

3.3 存储层次与 K 循环

固定 Block (m, n) 后, 数据沿存储层次流动, C Tile 留在寄存器中沿 K 持续累加。GPU 的存储层次从外到内是:

1. **Global Memory (DRAM/HBM)**: 所有 SM 共享的大容量显存, 带宽高但延迟大。
2. **Shared L2**: 所有 SM 的共同缓存层。
3. **L1 Cache**: 硬件管理, SM 私有, 无一致性。
4. **Shared Memory**: 软件管理, 显式访问, SM 私有的高速 SRAM。
5. **Registers**: 每个 Thread 或集体所有, 速度最快。
6. **Tensor Core**: 执行矩阵乘累加的专用单元。

L1 Cache 由硬件自动管理, *Shared Memory* 由程序显式管理。用户可以手动控制片上高速 SRAM 来处理大量输入, 这是 GPU 编程区别于 CPU 的关键能力。

K 循环的每一步分三阶段: **FETCH** (异步搬入 $A_{m,k}$ 、 $B_{n,k}$, 从 GMEM 经 L2 到 SMEM) → **MMA** (等待 SMEM 就绪, SMEM 到 Register, Fragments 交给 Tensor Core, C Tile 留在 Registers) → **REPEAT + STORE** (计算 K_k 的同时预取 K_{k+1} , K 循环结束后 Epilogue 后处理并写回 GMEM)。

例 取 4 个 K 步 K_0, K_1, K_2, K_3 。Prologue 先异步把 K_0 的 A/B Tile 装入 SMEM stage 0, 此时计算端没有 ready 的数据。随后沿 K 迭代: load K_1 到 stage 1, 同时 compute K_0 用 stage 0; load K_2 到 stage 0, 同时 compute K_1 用 stage 1, 依此类推。搬运和计算就这样重叠起来。

3.4 Tensor Core 指令: MMA (Volta/Ampere)

Tensor Core 是 GPU 上专做矩阵乘累加的硬件单元。从 Volta 架构开始引入, 它的核心指令 `mma` 由一个 Warp 同步调用, 完成一次 $(M, N, K) = (16, 8, 8)$ 的小型 $D = A \times B + C$ 运算, 输入和输出都在寄存器里, 称为 **Fragment** (片段)。

```
mma.sync.aligned.m16n8k8.row.col.f32.f16.f16.f32
{d0,d1,d2,d3} , {a0,a1,a2,a3} , {b0,b1} , {c0,c1,c2,c3} ;
```

直觉 这条指令的含义是: 32 个 Lane 协同完成一个 $16 \times 8 \times 8$ 的矩阵乘加, A 是 16×8 行主序, B 是 8×8 列主序, $\frac{C}{D}$ 是 16×8 的累加器。每个 Lane 只持有结果的一小片段寄存器, 所谓 Fragment。这种“把一个大矩阵乘拆给 32 个 Thread 共享”的映射, 就是 **Fragment Mapping**。

要点: *Thread 0* 的 `a0` 对应 $A[0,0]$ 这个元素。32 个 Lane 通过精心设计的映射, 共同拼出整个小矩阵, 没有人持有完整的矩阵。这就是为什么 *Tensor Core* 编程比普通 *CUDA* 更复杂: 你要管理 *Fragment* 的布局。

3.5 WGMMA (Hopper)

到了 Hopper 架构 (SM90), *Tensor Core* 演进为 **WGMMA** (Warp Group MMA)。4 个 Warp (共 128 个 Thread, 称为 **WarpGroup**) 同步发起一次更大的 $(64, 256, 16)$ 矩阵乘, 然后异步执行:

```
wgmma.mma_async.sync.aligned.m64n256k16.f32.f16.f16 d, a-desc, b-desc, (ctrl-imms);
```

`.sync.aligned` 是“集体发起”, 不代表计算完成。流程是: `fence` → `mma_async` → `commit`, 然后 *WarpGroup* 可以继续准备下一批工作, *Tensor Core* 在后台异步跑; 最后用 `wait_group` 等计算完成, 再消费 C/D 结果。这类似于 CPU 启动 *Kernel* → GPU 运行 → CPU 同步的模式。

关键改进是 **Tensor Core** 可以直接从 **Shared Memory** 读入 A/B，不再需要先把数据装到寄存器 Fragment 里。这降低了寄存器压力，支持更大的 Tile。

三代对比：Volta/Ampere 的 `mma.sync` 依赖 *SIMT* 和 *scoreboard* 隐藏延迟，输入在寄存器；Hopper 的 *WGMMMA* 异步执行，显式等待，输入从 *SMEM* 直接读取，寄存器压力下降，支持更大的 *Tile*。

3.6 TCGen05 + TMEM (Blackwell)

Blackwell 架构 (SM100) 把演进推向新阶段：累加器离开了通用寄存器，搬进专用的 **TMEM** (Tensor Memory)，单个 Thread 即可发起 1-CTA 或 2-CTA 的异步 MMA：

```
tcgen05.mma.cta_group::{1|2} [d-tmem], a-desc, b-desc, idesc, (ctrl-imms);
```

直觉 演进脉络是“发起粒度越来越大、专用性越来越强”：Volta/Ampere 用 1 个 Warp 发起，Hopper 用 1 个 WarpGroup (4 个 Warp)，Blackwell 只需 1 个 Thread 发起，甚至能让一对 SM (2-CTA) 协作完成一次更大的 MMA。

SIMT 仍是 *CUDA* 的控制基础，但 *Tensor Core* 的核心执行已经从逐 *Lane* 的通用计算，演进为专用、异步、可独立调度的加速器。换句话说，GPU 内部已经分化出“控制 CPU” (*SIMT Threads* 做控制与搬运) 和“计算 DSP” (*Tensor Core* 专做矩阵乘) 两类角色。

4 数据搬运与排布

前面我们看到了 *Tensor Core* 的强大，但要把数据持续喂给它，才是性能优化的主战场。本章以 Ampere (SM80) 为例，涉及两次搬运：**GMEM** → **SMEM** (Global Memory 到 Shared Memory) 和 **SMEM** → **Register** (Shared Memory 到寄存器 Fragment)。

4.1 ldmatrix: SMEM → Register Fragment

`mma.sync` 的输入是寄存器 Fragment，但数据在 Shared Memory 里，需要一个 Warp 协作的加载指令把 SMEM 里的矩阵搬进各 Lane 的寄存器，这就是 `ldmatrix`：

```
ldmatrix.sync.aligned.m8n8.x1.shared.b16 {d0}, [addr];
```

直觉 `ldmatrix` 让 32 个 Lane 同步执行，加载一个 8×8 的 b16 矩阵，每个 Lane 得到 1 个 b32 (打包了两个 b16)。Lane 0 到 Lane 7 分别给出矩阵 8 行的起始地址，硬件据此从 SMEM 把整行数据广播给对应 Lane。变体 `.x1/.x2/.x4` 控制一次加载几个矩阵。

设计巧妙之处：每个 *Lane* 只需提供一行地址，就能集体加载整个矩阵，且 *Fragment* 映射与 *mma* 期望的布局对齐，省去了手动整理寄存器的麻烦。

4.2 Bank Conflict: Shared Memory 的隐藏陷阱

Shared Memory 不是一块简单的内存，它被组织成 32 个 Bank，每个 Bank 宽 4 字节。一个 Warp 的访存请求里，落在同一 Bank 的不同 word 必须分批服务，冲突数最多时一个请求要分 8 轮才能完成，这就是 **Bank Conflict** (Bank 冲突)。

例 考虑一个 FP16 矩阵 Tile，行步长 128 字节。Shared Memory 地址按 $\text{bank} = \frac{\text{byte_addr}}{4} \bmod 32$ 计算。128 字节恰好跨过 32 个 Bank 又回到 Bank 0，所以连续 8 行的同一列地址全落在 Bank 0，形成 8-way 冲突，需要 8 个 service round。

例外是：多个 Lane 读取同一个 word 时可以 broadcast，不按冲突处理。

关键认识：*ldmatrix* 的 Lane/寄存器映射可以完全正确，但它触碰的 SMEM 地址仍可能集中到同一组 Bank。所以光看指令对不对不够，还要管数据的物理布局。

4.3 XOR Swizzle: 打乱布局消除冲突

解决 Bank Conflict 的办法是 **Swizzle** (搅动)：在写入 SMEM 时，对每行的 sector 顺序做按行号异或的置换，让原本挤在同一组 Bank 的访问展开到全部 32 个 Bank。

直觉 直觉是这样：未 Swizzle 时，8 行的同一 logical sector 落到同 4 个 Bank，需要 8 个 service round (8-way 冲突)。做 128B XOR Swizzle 后，每行的 sector 顺序按行号置换，8 次请求均匀命中 32 个 Bank，只需 1 个 service round，零冲突。

形式上， $\text{physical_sector} = \text{logical_sector} \text{ XOR } \text{row}$ ，然后 $\text{bank} = 4 \times \text{physical_sector} + \text{word_offset}$ 。

这看似只是地址变换，但它是 GPU 矩阵乘 Kernel 能跑满带宽的关键技巧之一。Swizzle 模式被写进 *Tensor Map descriptor*，由硬件或编译器自动处理。

4.4 cp.async: 绕过中间寄存器

把数据从 Global Memory 搬到 Shared Memory，朴素做法是先 load 到寄存器再 store 到 SMEM，需要两条指令和一个中间寄存器。Ampere 引入 cp.async (或 LDGSTS) 让数据直接从 GMEM 进 SMEM，省去中间环节：

```
// 朴素做法: load -> register -> store
uint4 tmp = gmem[src];
smem[dst] = tmp;
__syncthreads();
```

```
// cp.async: 直接 GMEM -> SMEM
cp.async.cg.shared.global [dst], [src], 16;
cp.async.commit_group;
// compute on an older stage
cp.async.wait_group 0;
```

`cp.async` 减少显式 `load/store` 指令和中间寄存器；单个 `group` 用 `wait_group 0` 才能证明 `ready`。多 `stage` 时 `wait_group N` 可保留至多 N 个最新 `group` 未完成，让你在等数据的同时继续算老 `stage` 的数据。

4.5 Double Buffer: 空间换重叠

光有两个 `stage` 的存储空间，只是提供了“可以重叠”的场地；要真正重叠搬运与计算，还需要独立的 in-flight copy 与 compute，以及完整的 `ready/free` 协议。这就是 **Double Buffer** (双缓冲)：

1. **acquire**: 等某个 `stage` 已 `free` (可以被覆写)。
2. **produce**: 异步填充下一个 `K Tile` 到这个 `stage`。
3. **consume**: 等数据 `ready` 后执行 MMA。
4. **release**: 读完才允许后续覆写。

直觉 想象一条流水线：load K_0 到 `stage 0` 的同时，`stage 1` 在 compute K_1 ；下一轮 load K_2 到 `stage 1`，`stage 0` compute K_3 。搬运和计算始终错开一格，谁也不闲着。

4.6 TMA: 一个 Thread 搬运整个 Tile

Hopper 引入 **TMA** (Tensor Memory Accelerator)，单条指令就能发起整个 `Tile` 的搬运，一个 elected Thread 提交坐标，硬件执行多维地址生成、coalescing、可选的越界填充 (OOB fill) 和 Shared Memory swizzle：

```
cp.async.bulk.tensor.2d.shared::cta.global
.mbarrier::complete_tx::bytes
[smem], [tensor_map, {coord0, coord1}], [bar];
```

地址规则被放进一个不透明的 *Tensor Map descriptor*，包含 *base address*、*global shape*、*strides*、*box shape*、*element type*、*swizzle mode*。你只需提交坐标，硬件搞定剩下的多维寻址和 *swizzle*，大大简化了搬移代码。

4.7 mbarrier 与 Warp Specialization

异步搬运需要一套完成协议，Hopper 用 **mbarrier** (共享内存里的 8 字节对象) 来协调。它维护当前世代 (*generation*)、等待位 (*parity*) 和未到达者计数 (*pending_count*)，以及 TMA 待完成字节数 (*tx_count*)。完成条件是 `pending_count == 0` 且 `tx_count == 0`，随后世代推进、*parity* 翻转。

1. **init**: `mbarrier.init(bar, 1)`，初始化并建立可见性。
2. **expect**: `arrive.expect_tx(bytes)`，登记本轮待完成字节数。
3. **complete**: `TMA complete_tx(bytes)`，硬件报告 copy 完成。
4. **wait**: `try_wait.parity(bar, p)`，成功后 consumer 才读 SMEM。

基于 `mbarrier`，可以把控制流拆分成不同角色，这就是 **Warp Specialization** (线程束特化)：

1. **TMA producer**: 负责 `tile schedule`、`TMA issue`、`barrier arrive`。

2. **MMA consumer**: 等数据 ready, 持续发异步 MMA, 读完后发出 free。
3. **Writeback WG**: 等累加器, 执行 Epilogue 和写回。

直觉 这就像工厂里的分工: 一组工人专门搬运原料 (producer), 一组专门加工 (consumer), 一组专门打包成品 (writeback)。它们用 mbarrier 互相发信号, 谁也不阻塞谁, 搬运和计算完全重叠。

同样是“异步”, 三代 *Tensor Core* 的发起粒度、地址描述、消费者接口和完成协议都不同。不要把 *cp.async*、TMA、WGMMA 当作同一条指令的改名: SM80 用 *cp.async + ldmatrix + mma.sync*; SM90 用 TMA + WGMMA descriptor 直接消费 SMEM; SM100 用 TMA + TCGen05, 累加器进入 TMEM。

5 用 TileLang 表达分块矩阵乘

手写 CUDA 来组合搬运、Swizzle、异步流水线和 Tensor Core 指令极其繁琐。TileLang 是一种领域专用语言, 让你用 Tile 操作来表达分块计算, 把常规的 thread mapping 和指令 lowering 交给编译器。

下面是一个 **Level-2 GEMM Kernel**: 一个 CTA 负责一个 128×256 的 C Tile, 沿 *K* 方向每次消费 64, 用 4 级流水线重叠搬运与计算。

```
@tilelang.jit(target={"kind": "cuda", "arch": "sm_90a"})
def gemm_level2(A, B):
    M, N, K = T.const("M, N, K")
    A: T.Tensor((M, K), T.bfloat16)
    B: T.Tensor((K, N), T.bfloat16)
    C = T.empty((M, N), T.bfloat16)
    BM, BN, BK = 128, 256, 64

    with T.Kernel(N // BN, M // BM, threads=256) as (bx, by):
        A_shared = T.alloc_shared((BM, BK), T.bfloat16)
        B_shared = T.alloc_shared((BK, BN), T.bfloat16)
        C_fragment = T.alloc_fragment((BM, BN), T.float32)
        T.use_swizzle(panel_size=4)

        T.clear(C_fragment)
        for ko in T.Pipelined(K // BK, num_stages=4):
            T.copy(A[by * BM, ko * BK], A_shared)
            T.copy(B[ko * BK, bx * BN], B_shared)
            T.gemm(A_shared, B_shared, C_fragment)

        C_shared = T.alloc_shared((BM, BN), T.bfloat16)
        T.copy(C_fragment, C_shared)
        T.copy(C_shared, C[by * BM, bx * BN])
    return C
```

逐行批注: @tilelang.jit 指定目标硬件为 sm_90a (Hopper)。T.const("M, N, K") 声明符号化维度。BM, BN, BK = 128, 256, 64 定义 Tile 大小。T.Kernel(N // BN, M // BM, threads=256) 声明 Grid 大小和每个 Block 的 Thread 数, bx, by 是 Block 坐标。T.alloc_shared 在 Shared Memory 分配 A/B Tile, T.alloc_fragment 在寄存器分配 C 累加器。T.use_swizzle(panel_size=4) 启用 Swizzle 以避免 Bank Conflict。T.clear(C_fragment) 清零累加器。T.Pipelined(K // BK, num_stages=4) 表达沿 K 方向的 4 级流水线。循环体里 T.copy 把 GMEM 的 A/B 搬进 SMEM, T.gemm 让 Tensor Core 做 SMEM 到 fragment 的矩阵乘累加。K 循环结束后, C_fragment 经 SMEM 写回 GMEM。

对比手写 CUDA, TileLang 把 Swizzle、异步搬运、流水线同步都抽象成 T.copy、T.gemm、T.Pipelined 几个原语, 代码量从几百行压缩到二十几行。

6 本章你将学会

1. 解释 SIMT 硬件 (SM、Warp、Thread、Shared Memory) 与 CUDA 编程模型 (Grid、Block、Warp、Thread) 的层级对应关系。
2. 写出一个完整的 CUDA Vector Add 程序, 包括 cudaMalloc、cudaMemcpy、Kernel launch 与 cudaDeviceSynchronize, 并解释 Kernel launch 的异步性。
3. 识别 Memory Coalescing 与 Warp Divergence, 说出它们如何影响性能以及如何避免。
4. 描述 Tensor Core 三代演进 (MMA → WGMMA → TCGen05+TMEM) 在发起粒度、异步性和存储位置上的区别。
5. 解释 GMEM 到 Tensor Core 的数据路径上, Shared Memory Bank Conflict、XOR Swizzle、cp.async、Double Buffer、TMA 与 mbarrier 各自解决什么问题。
6. 读懂 TileLang 的 Level-2 GEMM Kernel, 指出 T.copy、T.gemm、T.Pipelined 分别对应搬运、计算和流水线。

7 要点速查

概念	英文	一句话要点
线程束	Warp	32 个 Thread 共享指令流, SIMT 的基本单位
合并访存	Memory Coalescing	相邻 Thread 访问相邻地址, 合并成少量 32B transaction
分支分歧	Warp Divergence	Warp 内不同路径需依次执行, 浪费算力
共享内存	Shared Memory	SM 私有高速 SRAM, 软件显式管理
Bank 冲突	Bank Conflict	多 Lane 命中同 Bank 需分批服务
搅动布局	XOR Swizzle	行号异或置换 sector, 消除 Bank Conflict
异步拷贝	cp.async	GMEM 直达 SMEM, 绕过中间寄存器

双缓冲	Double Buffer	acquire/produce/consume/release 协议，重叠搬运与计算
张量内存加速器	TMA	单 Thread 提交坐标，硬件搬运整个 Tile
内存屏障	mbarrier	expect_tx/complete_tx/try_wait 完成协议
线程束特化	Warp Specialization	producer/MMA consumer/writeback 分工
矩阵乘累加指令	mma/WGMMA/TCGen05	Warp→WarpGroup→Thread，三代 Tensor Core 演进
张量内存	TMEM	Blackwell 把累加器移出通用寄存器到专用存储

8 小结

我们从最朴素的 Vector Add 走到了现代 GPU 的分块矩阵乘，贯穿四条主线。第一，SIMT 编程模型用 Grid、Block、Warp、Thread 把问题按层级拆分，优化时要关注访存合并与分支分歧。第二，Tiled GEMM 让一个 Block 负责一个 C Tile，沿 K 反复累加，Tensor Core 从 Volta 的 Warp 级 MMA，演进到 Hopper 的 WarpGroup 级 WGMMA，再到 Blackwell 的单 Thread 发起 TCGen05 与专用 TMEM。第三，数据搬运是性能的主战场，Shared Memory 的 Bank Conflict 需要 XOR Swizzle 化解，cp.async 与 TMA 实现异步搬运，Double Buffer 与 mbarrier 配合 Warp Specialization 让搬运与计算完全重叠。第四，TileLang 用 T.copy、T.gemm、T.Pipelined 把这些底层细节抽象成简洁的 Tile 原语，让高性能 Kernel 从数百行手写 CUDA 压缩到二十几行。

一句话总结：**高性能 GPU Kernel = 计算分块 + 数据布局 + 搬运流水线 + 正确同步**。下一章我们将从单卡 GPU 走向多卡分布式训练，看看大模型如何在这些并行单元上协同训练。

讲义基于 HPC101 课程「GPU Programming」内容编写